AWS PROJECT

Production-Style 3-Tier Web Architecture on AWS

Secure | Scalable | Highly Available

Adhithyan Sivaraman T

Cloud Infrastructure Engineer

Region: ap-south-1 (Mumbai)

VPC + Subnets

Network Isolation

EC2 (3 Tiers)

Web · App · Bastion

RDS MySQL

Managed Database

ALB + Routing

Path-based traffic

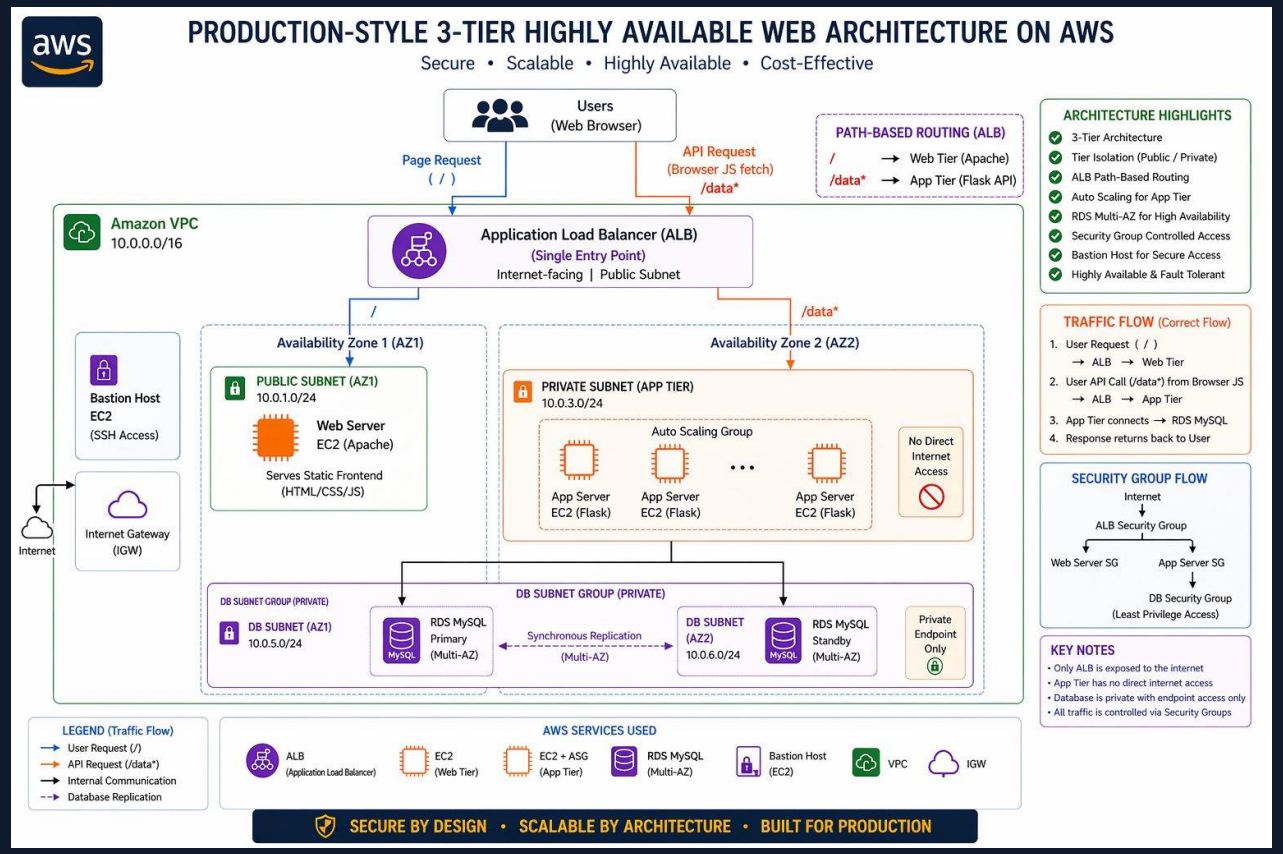
Auto Scaling

High Availability

ARCHITECTURE

System Architecture Overview

Multi-tier isolation with path-based routing



Request Flow

1. User sends request from browser
2. Request reaches Application Load Balancer
3. ALB routes: / → Web Tier | /data → App Tier
4. App Tier connects to RDS MySQL
5. Response returns via ALB to browser

Security Design

Only ALB is exposed to the internet
App Tier and Database in private subnets
No direct browser access to backend
Security groups enforce least-privilege access

Key Insight

ALB acts as the single entry point enabling secure and scalable communication between the public internet and private backend tiers.

GOLD CONCEPT

How the Browser Reaches the Backend

The #1 interview concept — and the real problem this architecture solves

✕ WRONG

Browser (Public IP)

→ App Server (Private IP)

🚫 **BLOCKED — Private subnet, no public access**

✓ CORRECT

Browser

→ ALB (Public)

→ App Tier (Private)

✓ **DB Connection Successful!**

Why This Matters

Private subnet has no public exposure

App SG does not allow internet traffic — browser requests come from public IPs and are rejected.

ALB acts as trusted internal caller

ALB lives inside the VPC. App SG allows ALB SG — so traffic flows: Browser → ALB (public) → App (private).

API calls originate from browser JS

The frontend fetch() in JavaScript runs in the user's browser — not from the web server. This is why ALB must be in the path.

NETWORKING

VPC Architecture & Network Design

Custom VPC: 10.0.0.0/16 | Region: ap-south-1 | 2 Availability Zones

PUBLIC SUBNET

AZ1 + AZ2

- Bastion Host
- Web Server
- NAT Gateway
- ALB

PRIVATE APP SUBNET

AZ1 + AZ2

- App Server (Flask)
- Auto Scaling Group
- Flask :8000

PRIVATE DB SUBNET

AZ1 + AZ2

- RDS MySQL
- DB Subnet Group
- Multi-AZ Replica

- Custom VPC prevents default network exposure
- Private subnets = no public IP, no direct internet access
- NAT Gateway enables private outbound (pip installs etc.)
- Security Group chaining = least-privilege between tiers

⚠ **Key Concept:** Browser requests cannot directly reach private subnets — all access must go through ALB.

WEB TIER

Frontend: Web Tier (Apache + HTML/JS)

Public Subnet | EC2 t2.micro | Apache HTTP Server

Static HTML Page

Hosted via Apache on EC2 in public subnet. Serves the landing page at /

Test Backend Button

JavaScript fetch() calls the ALB DNS + /data endpoint on click

Never Calls Backend Directly

Frontend JS always routes through ALB — backend is in private subnet

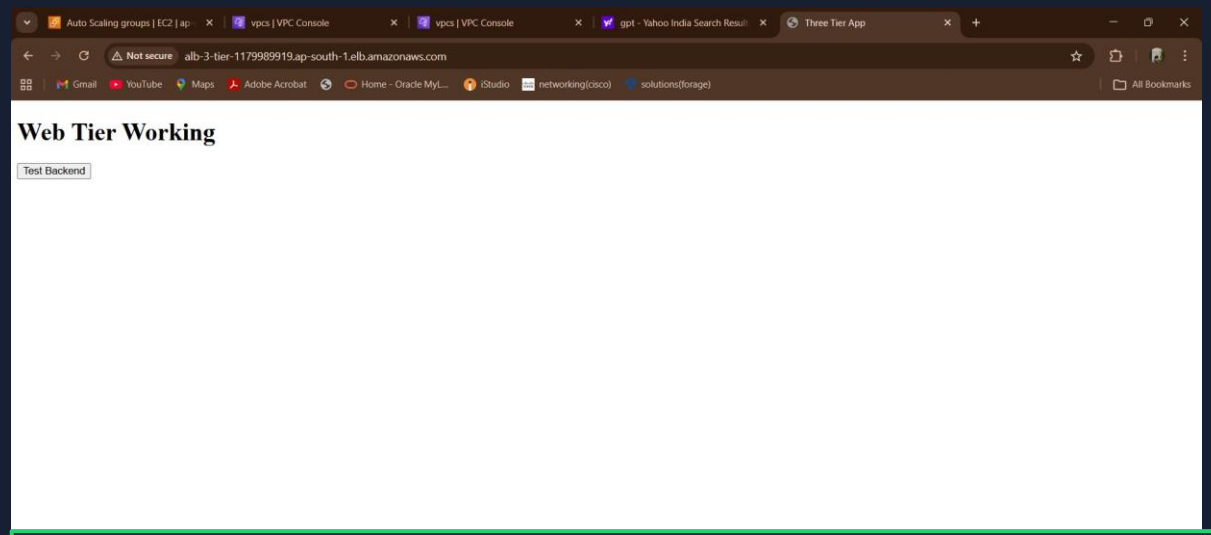
Apache Auto-starts

systemctl enable httpd — survives reboots

JavaScript (Frontend) — index.html

```
fetch("http://ALB-DNS/data")
  .then(r => r.text())
  .then(d => { output.innerText = d; });
```

ALB DNS Working — Web Tier Live



How It Works

Button triggers JavaScript fetch() function
fetch() sends GET request to ALB at /data endpoint
ALB path rule routes /data to App Tier backend
Response is displayed dynamically in the browser

Key Insight: API calls originate from browser JS, not the web server. The web server only serves static HTML/JS files.

APP TIER

Backend: App Tier (Flask API)

Private Subnet | Port 8000 | Python Flask | MySQL Connector

Web → App Connectivity (curl)

Web → App Connectivity: Confirms EC2 Web Tier can reach the Flask App Tier on port 8000, validating security group rules.

App → DB Connection (DB connection successful!)

Backend API Logic (Flask — app.py)

```
@app.route("/data")
def get_data():
    conn = mysql.connector.connect(
        host="RDS-ENDPOINT",  user="admin",
        password="***",  database="dbname")
    return "DB connection successful"
```

This endpoint is triggered via ALB path routing when the browser calls /data.

Python Flask

Lightweight REST framework. Endpoint /data handles DB queries.

host=0.0.0.0

Binds to ALL interfaces — required for ALB to reach it. Does NOT expose publicly.

Port Matching (Critical)

Target group port MUST = app port. Flask = 8000. Mismatch → unhealthy targets → 502 errors.

Explanation

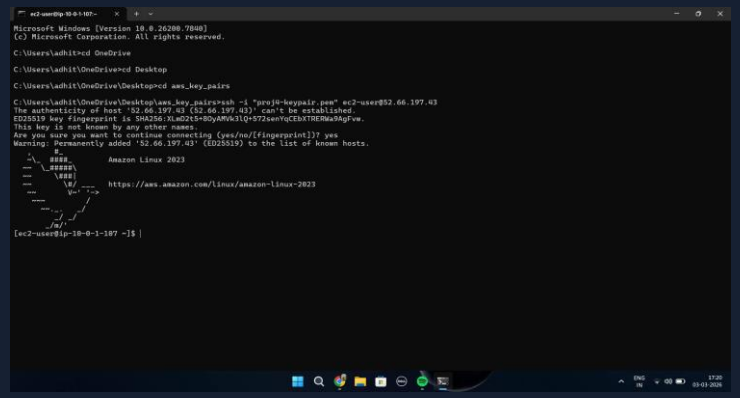
Flask receives the /data request forwarded by ALB
Backend connects to RDS MySQL via private endpoint
Returns response to frontend through ALB chain
Key Insight: Backend runs in private subnet, accessed only via ALB. No direct internet exposure.

SECURITY

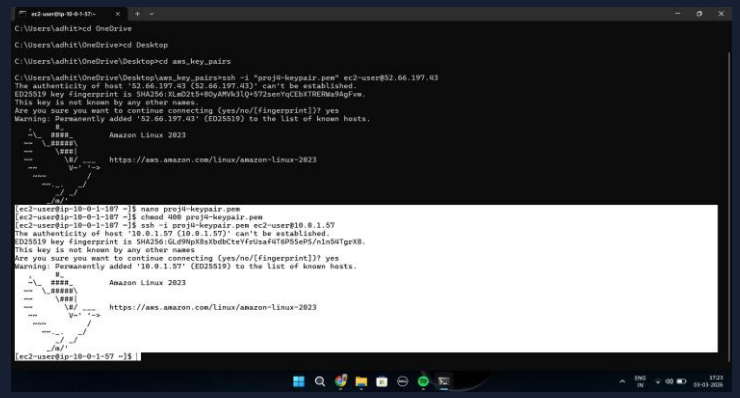
Bastion Host: Secure SSH Access Layer

Zero-trust network access — no direct exposure to private instances

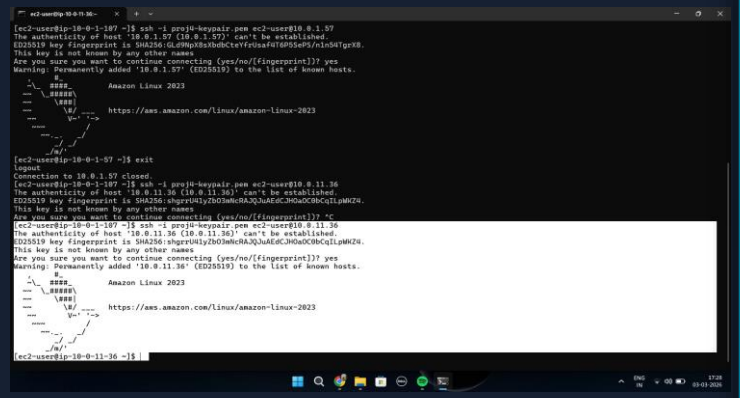
Local Machine → Bastion Host → Web/App EC2



① Bastion SSH Success
From local machine → Bastion (52.66.197.43)



② Bastion → Web Server
BastionHost jumps to Web EC2 (10.0.1.57)



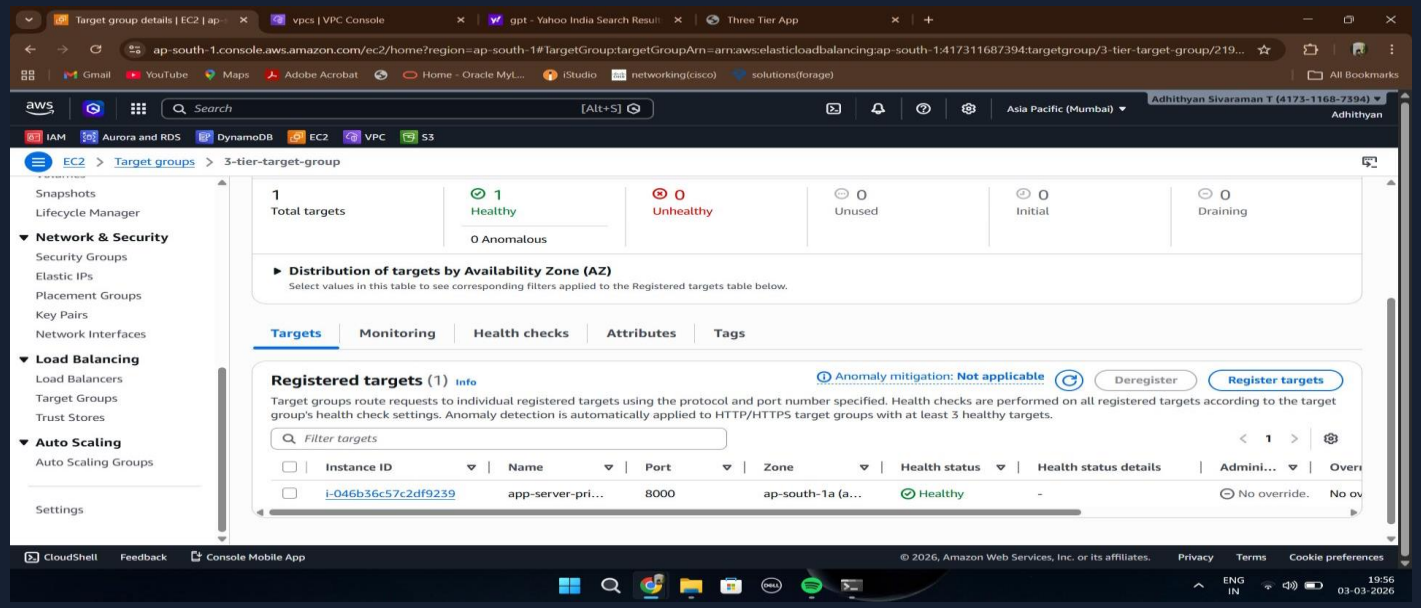
③ Bastion → App Server
BastionHost jumps to App EC2 (10.0.11.36)

LOAD BALANCER

Application Load Balancer & Health Checks

ALB as single public entry point — routes, balances and health-checks each tier

ALB Target Group — App Server Healthy on Port 8000



Target group shows 1 Healthy target on port 8000 — zero unhealthy

Web Target Group

Port: 80 | Health Path: /

Healthy

App Target Group

Port: 8000 | Health Path: /data

Healthy

☆ CRITICAL INTERVIEW TRAP

Target group port MUST match the app's listening port.

Apache = 80 | Flask = 8000

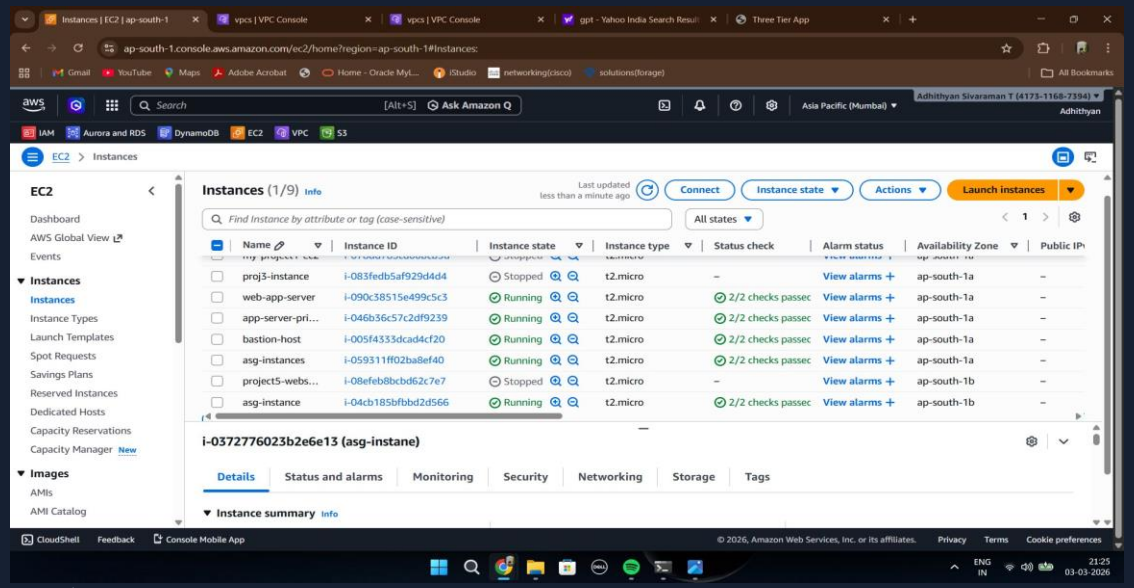
Mismatch → Unhealthy targets → 502 errors

HIGH AVAILABILITY

Auto Scaling Group & Multi-AZ Architecture

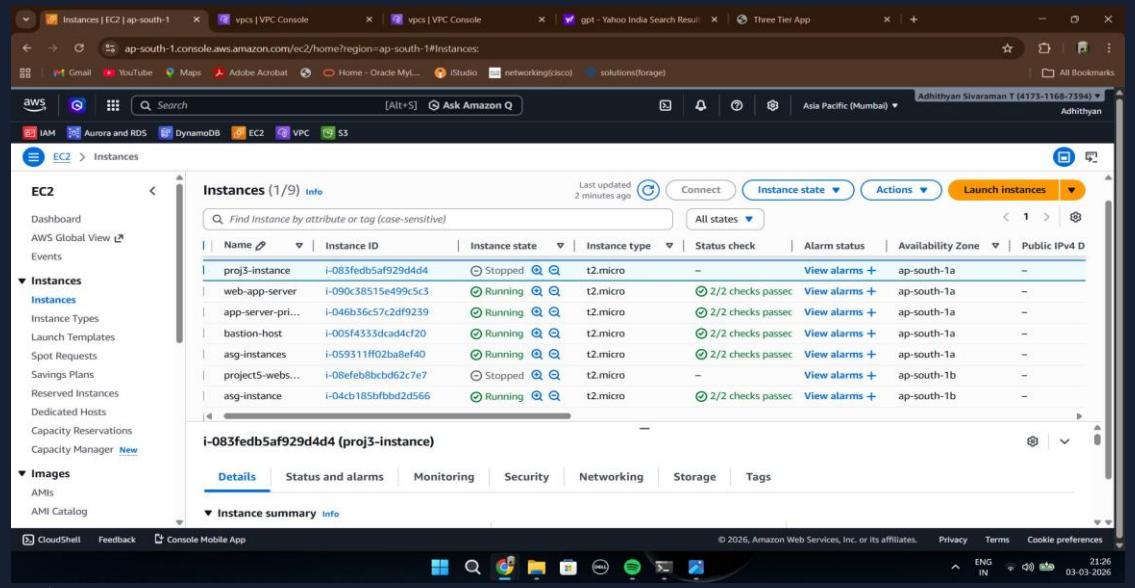
Self-healing, horizontally scalable app tier across 2 Availability Zones

ASG Instances Running (Multi-AZ)



ASG Running: Confirms Auto Scaling Group is active, managing instances across multiple AZs for fault tolerance and scalability.

Multi-AZ Proof — Instances in Different AZs



Multi-AZ Proof: Instance list confirms deployment across both Availability Zones, eliminating single points of failure.

Launch Template

AMI + SG + user-data
(NO subnet — controlled by ASG)

Desired: 2 Instances

ASG maintains count
automatically heals failures

Multi-AZ Subnets

Private-App-AZ1 +
Private-App-AZ2

ALB Health Checks

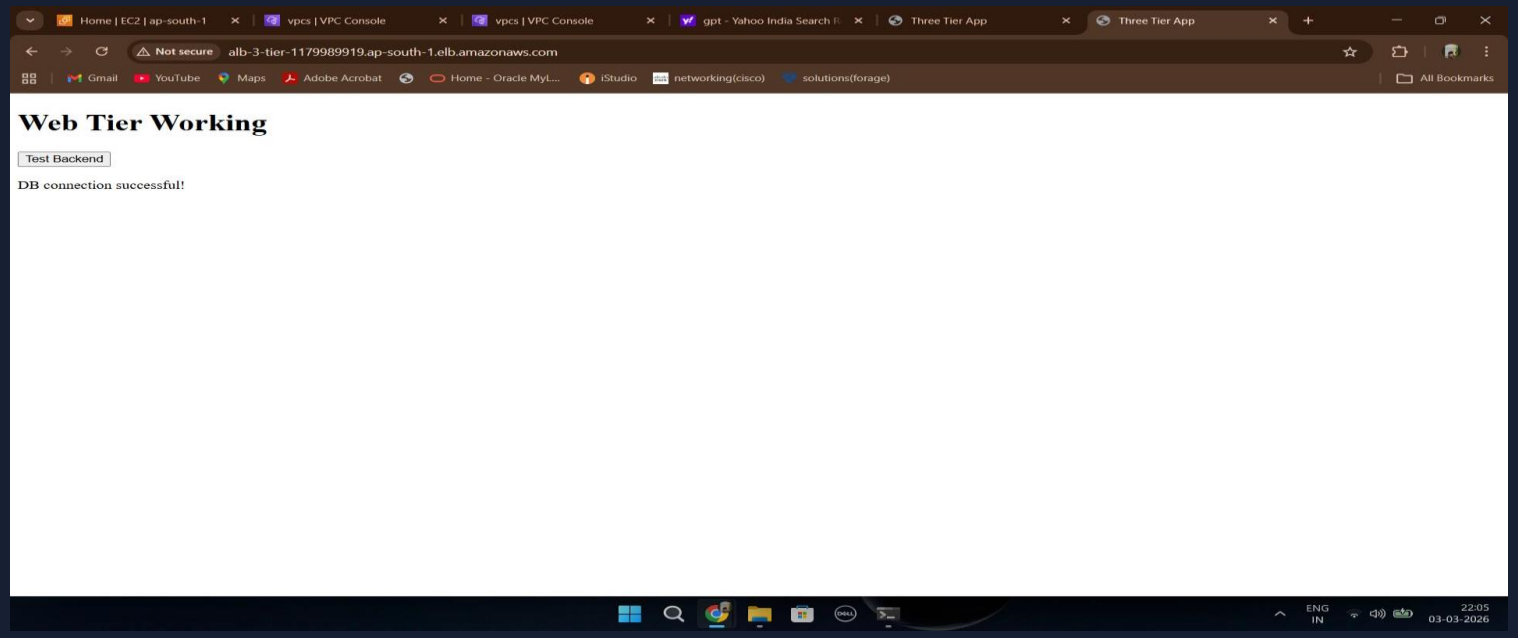
Unhealthy instances
deregistered automatically

FINAL RESULT

End-to-End: It Works!

Browser → ALB → Web → App → RDS MySQL → "DB connection successful!"

Final End-to-End Success



Problems Solved

✗ Browser couldn't reach backend

→ **Fixed via ALB routing**

✗ Misunderstood fetch() origin

→ **Browser JS → ALB (not web server)**

✗ Target group unhealthy (502)

→ **Fixed port 8000 mismatch**

✗ Flask not accessible via ALB

→ **host=0.0.0.0 binding**

End-to-End Data Flow

1. User clicks "Test Backend" button
2. Browser JS sends GET request to ALB (/data)
3. ALB path rule routes /data to App Tier
4. Flask processes the incoming /data request
5. Backend connects to RDS MySQL (private subnet)
6. RDS returns query result to Flask
7. Response travels back via ALB to browser
8. Browser displays: "DB connection successful!"

Why This Architecture Matters

Backend remains secure in private subnet
Enables independent scaling of App Tier via ASG
Prevents direct database exposure to internet
High availability ensured via Multi-AZ RDS
ALB provides controlled, path-based routing
Security groups enforce zero-trust tier isolation

INTERVIEW PREP

Key Concepts & Interview Q&A

Questions you'll be asked — and the real-world answers from this build

Q: Why can't browser access the private backend?

A: Backend EC2 is in a private subnet — no public IP. App SG blocks internet traffic. Solution: route through ALB (trusted VPC-internal component).

Q: Why ASG only on the App Tier?

A: App tier is compute-intensive and the scaling bottleneck. Web tier serves static content — lightweight. Scaling the right tier = cost efficiency.

Q: Why use a Bastion Host?

A: Provides controlled SSH access to private instances without exposing them to internet. Implements zero-trust access pattern.

Q: What ensures high availability?

A: Multi-AZ subnets + Auto Scaling Group + ALB health checks + stateless app design. Any AZ failure — ASG auto-replaces in the other AZ.

Q: ALB vs ASG — what's the difference?

A: ALB distributes incoming traffic across healthy instances. ASG manages the instance count, scaling and self-healing. They work together.

Q: What does host=0.0.0.0 mean in Flask?

A: Binds to all network interfaces on the instance. Required so ALB can reach the app. Does NOT make it public — security groups control that.